Testing

Run and understand the tests.

For: Engineers

One chapter of the handbook. The full handbook, with every chapter, is handbook.pdf in the same folder.

Tests answer one question: **does the code do what it claims?** Whether the *models* find the right policies is a separate question, answered by evaluation.md.

Levels

Level	Command	Needs	Proves
1. Unit + flow	`pytest -q` (in `policy_api/`)	Python only	Parsing, diff, candidate search, verdict handling, chat answer, Excel, API auth and flow
2. Postgres	`TEST_DATABASE_URL=… pytest -q`	Docker	The real store: SQL, pgvector search, versions, reference graph, audit rows, restart handling
3. Stack (demo)	`DEMO_MODE=1` + Compose	Docker	Images build, services start, proxy rules, download links, network containment
4. Stack (real models)	Smoke test below	GPU server	vLLM and TEI wiring, structured output, tool calling in Open WebUI
5. Evaluation	`python -m app.evaluate`	GPU server + labelled cases	Whether it finds the right policies (evaluation.md)

Run levels 1–2 on every change. Run 3 before every release. Run 4–5 after any model, image or prompt change.

Level 1: unit and flow tests

```
cd policy_api
python3 -m venv .venv && .venv/bin/pip install -r requirements-dev.txt
.venv/bin/python -m pytest -q
```

File	Covers
`test_documents.py`	Clause splitting (headings, list items, duplicate numbers), reference and defined-term extraction, front matter, diff (modified, added, removed, renumbered)
`test_pipeline.py`	The full sample run with stand-in models: which policies are Yes/Maybe/not affected, evidence, counts, the chat answer, workbook sheets, **policy text can't become an Excel formula**
`test_api.py`	API key required, **wrong Host header refused** (DNS-rebinding guard), lookups, upload → run → download, tampered download link refused, helpful errors

The stand-in models in `app/fakes.py` are deterministic. The fake judge says "yes" when a linked clause still contains wording the head policy removed, "unsure" when a clause is linked but has no such wording, and "no" otherwise. That makes flow tests exact. It does **not** make them evidence that the real models work.

Level 2: Postgres

```
docker run -d --name sk-test-db -e POSTGRES_PASSWORD=test -p 127.0.0.1:55432:5432 \
  -v "$PWD/../db/init.sql:/docker-entrypoint-initdb.d/init.sql:ro" pgvector/pgvector:pg17
TEST_DATABASE_URL=postgresql://postgres:test@localhost:55432/postgres .venv/bin/python -m pytest
-q
docker rm -f sk-test-db
```

`test_pgstore.py` truncates its tables, so never point `TEST_DATABASE_URL` at a real database.

Level 3: the stack in demo mode

Uses the sample policies and stand-in models, so no GPUs are needed. Make a throwaway env file (secrets from `openssl rand -hex 32`, any image tags, and `DEMO_MODE=1`), then:

```
docker compose --env-file /tmp/test.env up -d --build db policy-api proxy
curl -s http://127.0.0.1:8100/health                        # {"ok":true,"demo_mode":true}
curl -s -o /dev/null -w "%{http_code}\n" http://127.0.0.1:8100/subsets   # 404: API routes not
exposed
COMPOSE_ARGS="--env-file /tmp/test.env" scripts/check_containment.sh     # PASS contained
docker compose --env-file /tmp/test.env exec policy-api python -m app.evaluate
/eval/cases.example.json --demo
docker compose --env-file /tmp/test.env down -v
```

Testing the Open WebUI tool without Open WebUI

This applies to the `main` and `langgraph` versions. On the hermes version the Hermes plugin is tested by `policy_api/tests/test_hermes_plugin.py`, which runs in the normal test suite. The tool's logic can be run inside the policy-api container, which has the same libraries:

```
docker cp openwebui/policy_impact_tool.py secretary-keeper-policy-api-1:/tmp/tool.py
docker compose exec -T -e KEY=<POLICY_API_KEY> policy-api python3 - <<'EOF'
import asyncio, os, sys; sys.path.insert(0, "/tmp")
from tool import Tools
t = Tools(); t.valves.API_KEY = os.environ["KEY"]
files = [{"name": "POL-007-v4.md", "file": {"path": "/samples/changes/POL-007-v4.md"}}]
print(asyncio.run(t.check_policy_change("Safety", __files__=files)))
EOF
```

Also try: an unknown group, an ambiguous policy name, no file and no policy, and a wrong `API_URL`. Each must return a plain instruction for the chat model, never a stack trace.

Level 4: smoke test with the real models

On the GPU server, after setup.md:

1. From inside the network, `docker compose exec policy-api python3 -c "import httpx; print(httpx.get('http://granite-gateway:8000/v1/models').json())"` lists `granite-4.2-30b` (the gateway adds the key).
2. Ingest `samples/policies`, upload `samples/changes/POL-007-v4.md` in Open WebUI, and ask about Safety.
3. Expect SAF-112 and SAF-130 flagged, and SAF-141 and SAF-160 not affected. SAF-150 may be Yes or Maybe.
4. In `judgements`, every `result` has all six fields, and none has the "could not be read" reason.
5. Ask something ambiguous ("check the lockout thing") and confirm the assistant asks one question back.

Then ingest a copy of your real policies **in a test database** and run the evaluation.

Testing the tests

A test that can't fail proves nothing. When you add or change a check, break the thing it guards once and confirm the check fails:

- The containment script must fail when a service *can* reach the internet. During development it did: Docker Desktop ignored the "no outbound NAT" network setting, which is why policy-api and Open WebUI now sit only on the sealed `backend` network.
- It must also fail when it checked nothing. An early version printed "PASS" because it couldn't read the env file and skipped every service. It now fails if zero services were checked, and each check starts with a positive control (reaching an internal service).
- The formula-injection test checks the cell's *type*, not just its text.